PHASE 5: TESTING AND EVALUATION

5.1 Testing Approach

Testing was carried out to check whether the LMS works correctly and meets its requirements. The testing included unit testing, integration testing, system-level testing and negative testing.

The tests focused on user authentication, access control, course management, assignment creation, submissions, resubmissions, late penalties, marking, moderation and released results. Both valid and invalid inputs were considered.

5.2 Test Environment

The system was tested using:

- **Operating system:** Kali Linux
- **Programming language:** C++17
- **Database:** SQLite
- **Build tool:** CMake
- **Testing framework:** GoogleTest
- **Coverage tool:** gcovr

The application and automated tests were built and run from the project directory using the terminal.

5.3 Test Execution and Results

5.3.1 Unit Testing

Unit tests were used to check individual parts of the system. These included password hashing, input validation, date calculations, late-penalty calculations, assignment rules and mark validation.

5.3.2 Integration Testing

Integration tests checked whether different parts of the system work together correctly. These included database operations, user authentication, submissions, marking, moderation and audit records.

5.3.3 System Testing

System-level tests checked important operations through the application. These included the full workflow through the menus, first administrator setup, student access and the marking and moderation process.

5.3.4 Automated Test Results

The project contains seven GoogleTest files with a total of 104 automated tests.

The tests were executed using:

```
ctest --test-dir build --output-on-failure
```

The results were:

Description	Result
Total tests	104
Tests passed	104
Tests failed	0
Pass rate	100%
Execution time	4.07 seconds

All 104 automated tests passed.

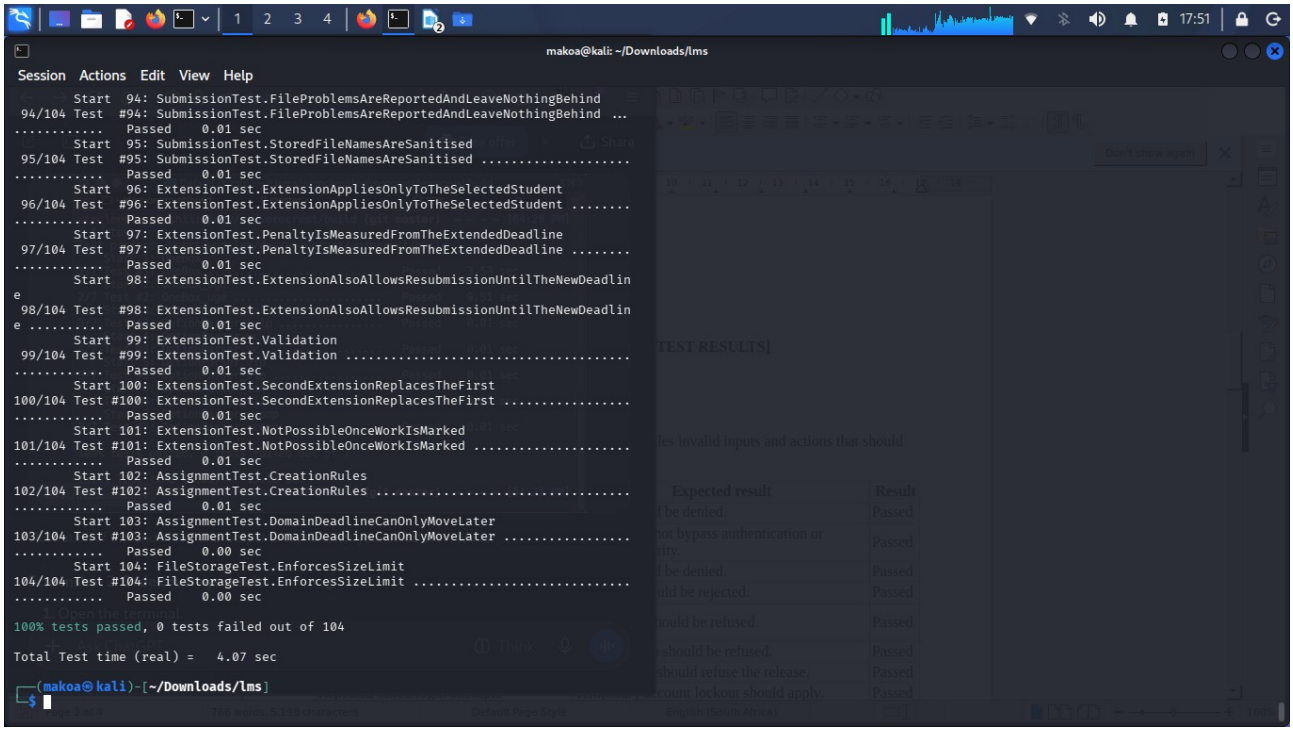


Figure 5.1: Automated test results.

5.4 Negative Testing

Negative testing was used to check how the system handles invalid inputs and actions that should not be allowed.

Test	Expected result	Result
Login with incorrect password	Access should be denied.	Passed
SQL injection attempt	Input should not bypass authentication or database security.	Passed
Student attempts staff functions	Access should be denied.	Passed
Mark above assignment maximum	The mark should be rejected.	Passed
Submission by a student not enrolled in the course	Submission should be refused.	Passed
Resubmission after the deadline	Resubmission should be refused.	Passed

Release of a mark without HoD approval	The database should refuse the release.	Passed
Repeated failed login attempts	Temporary account lockout should apply.	Passed

These cases are covered by the automated security, database, access-control, submission and moderation tests.

5.5 Acceptance Testing

The main acceptance requirements were checked through the automated tests and application workflow tests.

Acceptance requirement	Evidence from testing	Result
Users can log in securely.	Authentication and password tests	Passed
Users can only access permitted functions.	Access-control tests	Passed
Lecturers can create and manage assignments.	Assignment and course tests	Passed
Students can submit and resubmit work within the allowed period.	Submission tests	Passed
Late penalties are calculated correctly.	Late-penalty tests	Passed
Lecturers can mark submissions and record feedback.	Marking tests	Passed
HoDs can review and release marks.	Moderation tests	Passed
Students can only see released marks.	Moderation and released-results tests	Passed

A previously identified issue, DEF-01, was also manually retested. After the lecturer sent the mark for moderation and the HoD approved it, the student could see the released mark and feedback. The retest was successful.

5.6 Defect Management

Defects identified during testing were recorded and retested after the relevant changes.

DEF-01: Released mark not appearing for the student

- **Description:** An earlier manual test showed that a mark approved by the HoD was not appearing in the student's released results.
- **Severity:** High.
- **Status:** Resolved – manual retest passed.
- **Retest result:** The lecturer marked the submission and sent it for moderation. The HoD approved and released the mark. The student then viewed the mark and feedback successfully.

The manual retest confirmed that the marking, moderation and release workflow worked correctly.

5.7 Code Coverage and Evaluation

Code coverage was measured using GCC coverage tools and gcovr. The measurement focused on the service layer in the `src/services` directory.

The results were:

Measurement	Result
Executable lines	691
Lines executed	668
Line coverage	96.7%
Functions covered	54/54 (100%)
Branch coverage	821/1,393 (58.9%)
Line coverage target	70%

The service layer achieved 96.7% line coverage, which is above the 70% target. All 54 functions were covered. Branch coverage was 58.9%, showing that some decision paths still need further testing.

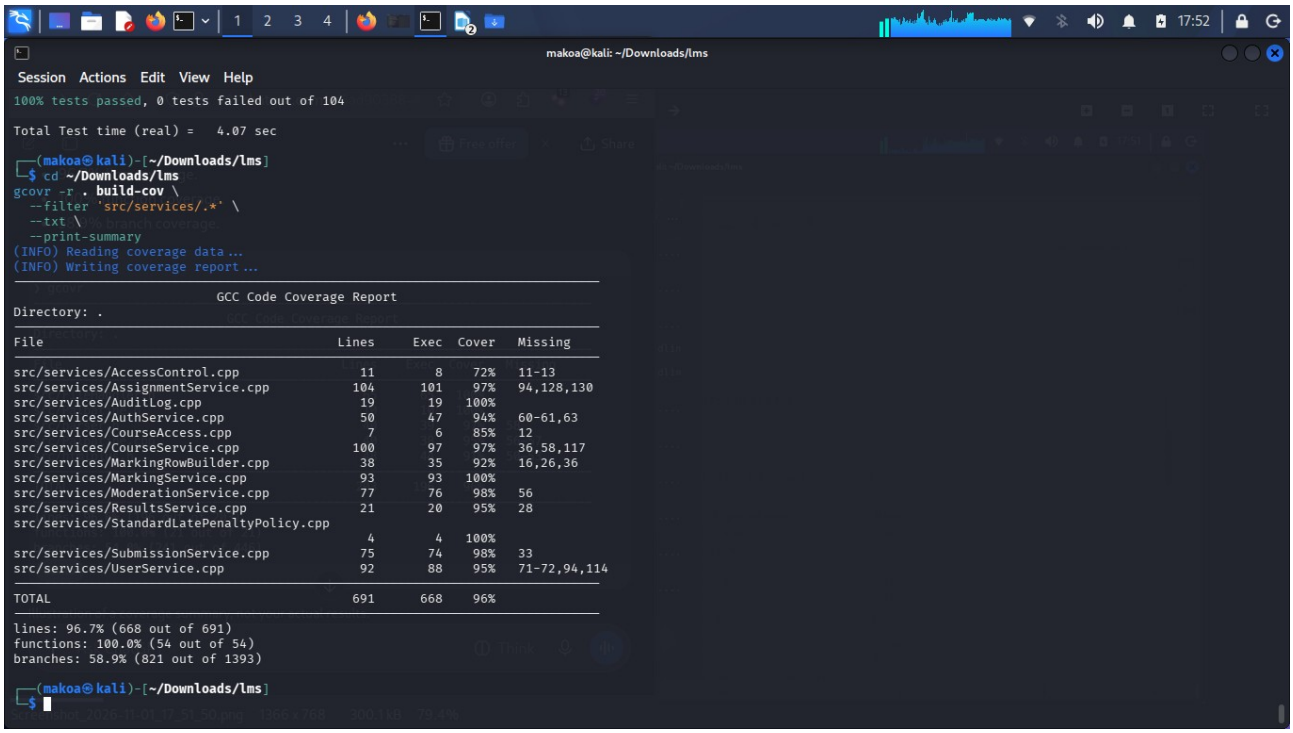


Figure 5.2: Service-layer code coverage results.

Overall, the testing results show that the main LMS functions work as expected. All 104 automated tests passed, and the service layer achieved the required line coverage target. Further testing can improve branch coverage and check more real-world usage situations.